COMP1314 Coursework

Cristian Grecov
cg7g25@soton.ac.uk

1 Relational Model

Ex 4

- Students(student firstname, student surname, student id, student email, address, contact number, module id, module name, leader, lecturer1, lecturer2, exam mark, coursework1, coursework2, coursework3)
- Faculties(faculty, building, room, capacity, lecturer email, lecturer firstname, lecturer surname)

Ex 5

The minimal Functional Dependencies are:

- Students:
  - student id → student firstname
  - student id → student surname
  - student id → student email
  - student id → year
  - student id → address
  - student id → contact number
  - module id → module name
  - module id → leader
  - module id → lecturer1
  - module id → lecturer2
  - student id, module id → exam mark
  - student id, module id → coursework1
  - student id, module id → coursework2
  - student id, module id → coursework3
- Faculties:

- lecturer email → lecturer firstname
- lecturer email → lecturer surname
- building, room → capacity
- building → faculty

Ex 6

The keys are:

- Students: student id, module id
- Faculties: lecturer email, building, room

Since on each record we have students and their respective marks for each module, student id and module id is the only candidate key that can be a primary key. The student id alone can give us information on student attributes like student firstname, student surname etc. The module id can only give information on module attributes like module name, leader etc. Only both can give this information but also information for each student's marks on each module.

With lecturer email we can only find lecturer first name and surname. With the building we can only determine the faculty it belongs to. With building and room, we can determine the capacity of any room in any building. Therefore, the only candidate key that gives us information about both the lecturer, buildings and rooms is the composite key (lecturer email, building, room) and suitable as primary key.

2 Normalisation

Ex 7

Students

For the Students relation the data is not in First Normal Form because we have repeating groups of the same attribute. For example, we have lecturer1 and lecturer2 columns. This is bad because if a new course appears with more than 2 lecturers, we have no way to represent that. We have the same issue with coursework1, coursework2, coursework3. Some courses don't even have 3 courseworks and coursework3 is empty. Also courses with more courseworks don't fit well with current structure.

To fix this we can create a separate relation:

Module_lecturers(module_id, lecturer_email).

Now we can eliminate repeated lecturer groups and easily add more lecturers in the Module_lecturers relation, for instance if a module has 3 lecturers. Furthermore, we can create another relation for coursework marks: Coursework_marks(student_id, module_id, coursework_number, mark). This way, we get rid of the repeated coursework sets in the Students relation and now we can

more easily add the marks for each student's coursework in each module in the new relation. This dynamically adapts to courses with less than or more than 3 courseworks.

After this the original Students relation would become Students(student first name, student surname, student id, student email, year, address, contact number, module id, module name, leader, exam mark).

Faculties

The faculties relation is in First Normal Form because it contains only atomic values (a single room, faculty, building etc. per cell). There are no lists of items inside 1 cell or repeated groups like room1, room2... Every row also identifies one unique assignment of a lecturer teaching in a specific room in a building. So, nothing needs to be changed to achieve First Normal Form.

Ex 8

Students

The Students relation before applying Second Normal Form is Students(student firstname, student surname, student id, student email, year, address, contact number, module id, module name, leader, exam mark).

It has the following partial-key dependencies:

- student id $\rightarrow$ student firstname
- student id $\rightarrow$ student surname
- student id $\rightarrow$ student email
- student id $\rightarrow$ year
- student id $\rightarrow$ address
- student id $\rightarrow$ contact number
- module id $\rightarrow$ module name
- module id $\rightarrow$ leader

We have 2 sets of partial-key dependencies: 1 depending on student id, the other depending on module id. Therefore, we can create separate relations for students, modules and remove them from the original relation. To avoid confusion the original relation should be renamed Exam_results as it contains information on student id, module id, and exam mark while the Students table will only contain student information.

After separating the student data and module data from the original relation into separate ones, this gives us the following relations:

- Students(student id, student firstname, student surname, student email, year, address, contact number)
- Modules(module id, module name, leader)
- Exam_results(student id, module id, exam mark)

For the Students relation the data types are student id: text (in case leading zeroes), student firstname: text, student surname: text, student email: text, year: integer, address: text, contact number: text (as it contains symbols like parantheses, +, -). The primary key is student id, which uniquely identifies each student and their attributes.

For the Modules relation the data types are module id: text, module name: text, leader: text. The primary key is module id which determines each unique module and its name, leader.

For the Exam_results relation the types are student id: text, module id: text, exam mark: real. The primary key is a composite key (student id, module id) identifying the exam mark for each student for each module.

When converting the data into First Normal Form we also created the relations:

- Module_lecturers(module_id, lecturer_email)
- Coursework_marks(student_id, module_id, coursework_number, mark)

For Module_lecturers the data types are module_id: text, lecturer_email: text and the primary key is the composite key (module_id, lecturer_email).

For Coursework_marks relation the data types are student_id: text, module_id: text, coursework_number: integer, mark: real. The primary key is the composite key (student_id, module_id, coursework_number).

These don't have any partial key dependencies and don't need to be decomposed into Second Normal Form as they already respect it, therefore we have now successfully decomposed the original Students table into Second Normal Form, and each attribute depends on a single unique key.

Faculties

The Faculties relation has the following partial-key dependencies:

- lecturer_email $\rightarrow$ lecturer_firstname
- lecturer_email $\rightarrow$ lecturer_lastname
- lecturer_email $\rightarrow$ faculty
- building, room $\rightarrow$ capacity

We can see 2 sets of partial-key dependencies, one for the lecturer_email key and one for the (building, room) composite key. And we can rename the Faculties relation Room_lecturers as after separating the partial-key dependencies it only contains building, room lecturer. It will store new records each time a lecturer teaches in a room. Therefore, we can create a separate relation for lecturers, and one for the rooms and their attributes.

This gives us the following relations:

- Lecturers(lecturer_email, lecturer_firstname, lecturer_surname, faculty)
- Rooms(building, room, capacity)
- Room_lecturers(building, room, lecturer_email)

The data types for the Lecturers relation are lecturer_email: text, lecturer_firstname: text, lecturer_surname: text, faculty: text. The primary key is lecturer_email, determining all other attributes for each unique lecturer.

For the Rooms relation the data types are building: text, room: text, capacity: integer. The primary key is a composite key (building, room) which uniquely identifies each room on the University of Northeasthampton's campus and its capacity.

For the Room_lecturers relation the data types are building: text, room: text, lecturer_email: text. The primary key is a composite key (building, room, lecturer_email) which identifies each new instance of a lecturer teaching in a room on campus.

Ex 9

Relation 1

After decomposing into Second Normal Form there are no transitive dependencies. All non-key attributes depend solely on the primary/composite key and on nothing else. Therefore, they are in Third Normal Form as well.

For the Students relation, student_firstname, student_surname, student_email, year, address, contact_number can only be determined using the primary key student_id.

In the Modules relation, module_name and leader depend only on the primary key module_id and can't be determined by anything else. In the Exam_results each new record of a student's exam result contains their id (student_id), the module the exam was for (module_id) and the mark (exam_mark). There are no non-key attributes and no transitive dependencies.

The Module_lecturers(module_id, lecturer_email) relation which we created during First Normal Form has no non-key attributes and thus no transitive dependencies.

The Coursework_marks relation created at the same time has composite key (student_id, module_id, coursework_number) and non-key attribute mark dependent only on this composite key.

Relation 2

This relation that we decomposed into Second Normal Form also doesn't have any transitive dependencies and it's already in Third Normal Form as well. No non-key attributes depend on anything other than the primary/composite key.

For the Lecturers relation, lecturer_firstname, lecturer_surname, faculty depend only on the primary key lecturer_email and on nothing else.

In the Rooms relation the capacity depends solely on the composite key (building, room) and can't be determined by other information.

In the Room_lecturers(building, room, lecturer_email) relation there are no non-key attributes so no transitive dependencies either.

3 Modelling

Ex 11

Table 1: Students

Attribute	Datatype
student_id	TEXT
student_firstname	TEXT
student_lastname	TEXT
student_email	TEXT
year	INTEGER
address	TEXT
contact_number	TEXT

Table 2: Modules

Attribute	Datatype
module_id	TEXT
module_name	TEXT
leader	TEXT

Table 3: Exam_results

Attribute	Datatype
student_id	TEXT
module_id	TEXT
exam_mark	REAL

Table 4: Module_lecturers

Attribute	Datatype
module_id	TEXT
lecturer_email	TEXT

Table 5: Coursework_marks

Attribute	Datatype
student_id	TEXT
module_id	TEXT
coursework_number	INTEGER
mark	REAL

Table 6: Lecturers

Attribute	Datatype
lecturer_email	TEXT
lecturer_firstname	TEXT
lecturer_surname	TEXT
faculty	TEXT

Table 7: Rooms

Attribute	Datatype
building	TEXT
room	TEXT
capacity	INTEGER

Table 8: Room_lecturers

Attribute	Datatype
building	TEXT
room	TEXT
lecturer_email	TEXT

4 Querying

Ex 13

```
SELECT
    students.student_id,
    students.student_firstname,
    students.student_lastname,
    AVG(exam_results.exam_mark) AS avg_exam_mark
FROM
    students
    JOIN exam_results ON students.student_id =
       exam_results.student_id
    JOIN modules ON exam_results.module_id =
       modules.module_id
WHERE
    modules.module_id LIKE 'COMP%'
```

```
    AND students.year = 1
GROUP BY
    students.student_id
ORDER BY
    avg_exam_mark DESC
LIMIT
    5;
```

Ex 14

```
SELECT
    module_id,
    leader_firstname,
    leader_surname,
    faculty,
    average_mark
FROM
    (
        SELECT
            modules.module_id,
            lecturers.lecturer_firstname AS
               leader_firstname,
            lecturers.lecturer_surname AS
               leader_surname,
            lecturers.faculty,
            AVG(coursework_marks.mark) +
               AVG(exam_results.exam_mark) AS
               average_mark,
            ROW_NUMBER() OVER (
                PARTITION BY
                    lecturers.faculty
                ORDER BY
                    (
                        AVG(coursework_marks.mark) +
                           AVG(exam_results.exam_mark)
                    ) DESC
            ) AS mark_rank_for_this_faculty
        FROM
            coursework_marks
            JOIN exam_results ON
               coursework_marks.module_id =
               exam_results.module_id
            JOIN modules ON exam_results.module_id =
               modules.module_id
```

```
            JOIN lecturers ON modules.leader =
               lecturers.lecturer_email
        GROUP BY
            exam_results.module_id
    )
WHERE
    mark_rank_for_this_faculty = 1;
```

Ex 15

```
SELECT
    module_lecturers.module_id,
    room_lecturers.building,
    room_lecturers.room
FROM
    exam_results
    JOIN module_lecturers ON exam_results.module_id =
       module_lecturers.module_id
    JOIN room_lecturers ON
       module_lecturers.lecturer_email =
       room_lecturers.lecturer_email
    JOIN rooms ON room_lecturers.building =
       rooms.building
    AND room_lecturers.room = rooms.room
GROUP BY
    module_lecturers.module_id,
    room_lecturers.building,
    room_lecturers.room
HAVING
    COUNT(DISTINCT exam_results.student_id) >
       rooms.capacity;
```